

Topic 4: Dynamic Programming

Design and Analysis of Algorithms Laboratory

Exercise 4.1: Dice Throw Problem

Aim:

To find the number of ways to achieve a target sum by throwing 'num_dice' each having 'num_sides' faces using dynamic programming.

Python Code:

```
def count_dice_ways(num_sides, num_dice, target):  
    """  
    dp[d][s] = number of ways to get sum s with d dice.  
    Time Complexity: O(num_dice * target * num_sides), Space: O(num_dice * target)  
    """  
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]  
    dp[0][0] = 1  
  
    for d in range(1, num_dice + 1):  
        for s in range(1, target + 1):  
            for face in range(1, num_sides + 1):  
                if s >= face:  
                    dp[d][s] += dp[d - 1][s - face]  
  
    return dp[num_dice][target]  
  
# Test Cases  
print("Test 1 (sides=6, dice=2, sum=7): ", count_dice_ways(6, 2, 7))  
print("Test 2 (sides=4, dice=3, sum=10):", count_dice_ways(4, 3, 10))
```

Sample Input:

Test Case 1: Sides = 6, Dice = 2, Target = 7

Test Case 2: Sides = 4, Dice = 3, Target = 10

Sample Output:

Number of ways to reach sum 7: 6

Number of ways to reach sum 10: 27

Exercise 4.2: Two-Line Assembly Line Scheduling

Aim:

To determine the fastest route through a 2-line factory assembly process considering station service times and transfer delays.

Python Code:

```
def assembly_line_scheduling(n, a1, a2, t1, t2, e1, e2, x1, x2):
    """
    Time Complexity: O(n), Space Complexity: O(1)
    """
    f1 = e1 + a1[0]
    f2 = e2 + a2[0]

    for i in range(1, n):
        new_f1 = min(f1 + a1[i], f2 + t2[i - 1] + a1[i])
        new_f2 = min(f2 + a2[i], f1 + t1[i - 1] + a2[i])
        f1, f2 = new_f1, new_f2

    return min(f1 + x1, f2 + x2)

# Sample Execution
n = 4
a1, a2 = [4, 5, 3, 2], [2, 10, 1, 4]
t1, t2 = [7, 4, 5], [9, 2, 8]
e1, e2 = 10, 12
x1, x2 = 18, 7
print("Min Time:", assembly_line_scheduling(n, a1, a2, t1, t2, e1, e2, x1, x2))
```

Sample Input:

n=4, a1=[4,5,3,2], a2=[2,10,1,4], t1=[7,4,5], t2=[9,2,8], e1=10, e2=12, x1=18, x2=7

Sample Output:

The minimum time required to process the product: 35

Exercise 4.3: Three-Line Assembly Line Scheduling with Dependencies

Aim:

To optimize job routing across 3 parallel assembly lines with station processing times and cross-line transfer costs.

Python Code:

```
def three_line_scheduling(stations, transfers):
    """
    stations: 3 x n matrix of times
    transfers: 3 x 3 transfer matrix between lines
    Time Complexity: O(n * L^2) where L=3, Space: O(L)
    """
    n = len(stations[0])
    dp = [stations[line][0] for line in range(3)]

    for j in range(1, n):
        next_dp = [float('inf')] * 3
        for curr_l in range(3):
            for prev_l in range(3):
                cost = dp[prev_l] + transfers[prev_l][curr_l] + stations[curr_l][j]
                if cost < next_dp[curr_l]:
                    next_dp[curr_l] = cost
            dp = next_dp

    return min(dp)

stations = [[5, 9, 3], [6, 8, 4], [7, 6, 5]]
transfers = [[0, 2, 3], [2, 0, 4], [3, 4, 0]]
print("Minimum Production Time:", three_line_scheduling(stations, transfers))
```

Sample Input:

Line 1: [5, 9, 3], Line 2: [6, 8, 4], Line 3: [7, 6, 5]
Transfers: [[0,2,3], [2,0,4], [3,4,0]]

Sample Output:

Optimal Scheduling Total Time: 21

Exercise 4.4: Minimum Path Distance in Matrix / TSP Tour

Aim:

To calculate the minimum path or cycle distance visiting all vertices in a given distance matrix.

Python Code:

```
import itertools

def min_cycle_distance(matrix):
    n = len(matrix)
    nodes = list(range(1, n))
    min_cost = float('inf')
    for perm in itertools.permutations(nodes):
        path = [0] + list(perm) + [0]
        cost = sum(matrix[path[i]][path[i+1]] for i in range(len(path)-1))
        if cost < min_cost:
            min_cost = cost
    return min_cost

# Test Cases
m1 = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
print("Test 1:", min_cycle_distance(m1))

m2 = [[0,10,10,10],[10,0,10,10],[10,10,0,10],[10,10,10,0]]
print("Test 2:", min_cycle_distance(m2))

m3 = [[0,1,2,3],[1,0,4,5],[2,4,0,6],[3,5,6,0]]
print("Test 3:", min_cycle_distance(m3))
```

Sample Input:

Matrix 1: { {0,10,15,20}, {10,0,35,25}, {15,35,0,30}, {20,25,30,0} }

Matrix 2: { {0,10,10,10}, ... }

Matrix 3: { {0,1,2,3}, ... }

Sample Output:

Output: 80

Output: 40

Output: 12

Exercise 4.5: 5-City Traveling Salesperson Problem (Held-Karp DP)

Aim:

To find the shortest TSP tour visiting 5 cities (A, B, C, D, E) with symmetric distances using Held-Karp dynamic programming.

Python Code:

```
def tsp_held_karp(dist_matrix, city_names):
    n = len(dist_matrix)
    memo = {}

    def get_min(mask, pos):
        if mask == (1 << n) - 1:
            return dist_matrix[pos][0], [0]
        if (mask, pos) in memo:
            return memo[(mask, pos)]

    ans = float('inf')
    best_p = []
    for nxt in range(n):
        if not (mask & (1 << nxt)):
            new_cost, p = get_min(mask | (1 << nxt), nxt)
            cost = dist_matrix[pos][nxt] + new_cost
            if cost < ans:
                ans = cost
                best_p = [nxt] + p

    memo[(mask, pos)] = (ans, best_p)
    return memo[(mask, pos)]

cost, path = get_min(1, 0)
full_path = [city_names[0]] + [city_names[i] for i in path]
return cost, " -> ".join(full_path)

cities = ['A', 'B', 'C', 'D', 'E']
# Symmetric matrix mapping given distances
dist = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 30],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 15],
    [25, 30, 20, 15, 0]
]
cost, route = tsp_held_karp(dist, cities)
print(f"Shortest Route: {route}, Total Distance: {cost}")
```

Sample Input:

Distances: A-B:10, A-C:15, A-D:20, A-E:25, B-C:35, B-D:25, B-E:30, C-D:30, C-E:20, D-E:15

Sample Output:

Shortest Route: A -> B -> D -> E -> C -> A, Total Distance: 85

Exercise 4.6: Longest Palindromic Substring

Aim:

To return the longest palindromic substring in s using DP or expand-around-center in $O(n^2)$ time.

Python Code:

```
def longest_palindrome(s):
    """
    Time Complexity:  $O(n^2)$ , Space Complexity:  $O(1)$ 
    """
    if not s:
        return ""
    start, max_len = 0, 1

    def expand(left, right):
        nonlocal start, max_len
        while left >= 0 and right < len(s) and s[left] == s[right]:
            length = right - left + 1
            if length > max_len:
                start = left
                max_len = length
            left -= 1
            right += 1

    for i in range(len(s)):
        expand(i, i) # Odd length
        expand(i, i + 1) # Even length

    return s[start:start + max_len]

# Test Cases
print(f's = "babad" -> "{longest_palindrome("babad")}"')
print(f's = "cbdd" -> "{longest_palindrome("cbdd")}"')
```

Sample Input:

```
s = "babad"
s = "cbdd"
```

Sample Output:

```
Output: "bab" (or "aba")
Output: "bb"
```

Exercise 4.7: Longest Substring Without Repeating Characters

Aim:

To find the length of the longest substring without repeating characters using sliding window DP.

Python Code:

```
def length_of_longest_substring(s):  
    """  
    Sliding window with hash map.  
    Time Complexity: O(n), Space Complexity: O(min(m, n))  
    """  
    char_map = {}  
    left = 0  
    max_len = 0  
  
    for right, ch in enumerate(s):  
        if ch in char_map and char_map[ch] >= left:  
            left = char_map[ch] + 1  
        char_map[ch] = right  
        max_len = max(max_len, right - left + 1)  
  
    return max_len  
  
# Test Cases  
print(length_of_longest_substring("abcabcbb"))  
print(length_of_longest_substring("bbbb"))  
print(length_of_longest_substring("pwwkew"))
```

Sample Input:

```
s = "abcabcbb"  
s = "bbbb"  
s = "pwwkew"
```

Sample Output:

```
Output: 3 ("abc")  
Output: 1 ("b")  
Output: 3 ("wke")
```

Exercise 4.8: Word Break Problem

Aim:

To determine whether string s can be segmented into a space-separated sequence of dictionary words.

Python Code:

```
def word_break(s, wordDict):
    """
    dp[i] represents whether s[:i] can be segmented.
    Time:  $O(n^2)$ , Space:  $O(n)$ 
    """
    word_set = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[len(s)]

# Test Cases
print(word_break("leetcode", ["leet", "code"]))
print(word_break("applepenapple", ["apple", "pen"]))
print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
```

Sample Input:

```
s = "leetcode", wordDict = ["leet","code"]
s = "applepenapple", wordDict = ["apple","pen"]
s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
```

Sample Output:

```
Output: true
Output: true
Output: false
```


Exercise 4.9: Word Break with Sentence Reconstruction

Aim:

To check segmentation against a custom dictionary and display the reconstructed words.

Python Code:

```
def word_break_custom(s, words):
    word_set = set(words)
    memo = {}

    def segment(sub):
        if not sub:
            return [""]
        if sub in memo:
            return memo[sub]
        res = []
        for w in word_set:
            if sub.startswith(w):
                rest = segment(sub[len(w):])
                for r in rest:
                    res.append((w + " " + r).strip())
        memo[sub] = res
        return res

    results = segment(s)
    return "Yes" if results else "No", results

dictionary = {"i", "like", "sam", "sung", "samsung", "mobile", "ice", "cream", "icecream", "man", "go", "mango"}
print("I like: ", word_break_custom("I like", dictionary)[0])
print("I likesamsung: ", word_break_custom("I likesamsung", dictionary)[0])
```

Sample Input:

Input: I like

Input: I likesamsung

Dict: {i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango}

Sample Output:

Output: Yes (Segmented as: "I like")

Output: Yes (Segmented as: "I like samsung" or "I like sam sung")

Exercise 4.10: Text Justification (Greedy / DP Word Packing)

Aim:

To format a sequence of words into fully justified lines of width `maxWidth` with balanced spacing.

Python Code:

```
def full_justify(words, max_width):
    res, cur, num_of_letters = [], [], 0
    for w in words:
        if num_of_letters + len(w) + len(cur) > max_width:
            for i in range(max_width - num_of_letters):
                cur[i % (len(cur) - 1 or 1)] += ' '
            res.append(''.join(cur))
            cur, num_of_letters = [], 0
        cur.append(w)
        num_of_letters += len(w)

    # Last line left-justified
    res.append(' '.join(cur).ljust(max_width))
    return res

words = ["This", "is", "an", "example", "of", "text", "justification."]
print(full_justify(words, 16))
```

Sample Input:

```
words = ["This", "is", "an", "example", "of", "text", "justification."], maxWidth = 16
```

Sample Output:

```
["This is an", "example of text", "justification. "]
```

Exercise 4.11: WordFilter: Prefix and Suffix Search

Aim:

To design a data structure that efficiently returns the highest index of a word matching both a given prefix and suffix.

Python Code:

```
class WordFilter:
    def __init__(self, words):
        self.map = {}
        for idx, word in enumerate(words):
            L = len(word)
            for i in range(L + 1):
                for j in range(L + 1):
                    pref = word[:i]
                    suff = word[j:]
                    self.map[(pref, suff)] = idx

    def f(self, pref, suff):
        return self.map.get((pref, suff), -1)

# Test Case
wf = WordFilter(["apple"])
print("WordFilter(['apple']).f('a', 'e') ->", wf.f("a", "e"))
```

Sample Input:

```
["WordFilter", "f"] -> [[["apple"]], ["a", "e"]]
```

Sample Output:

[null, 0]

Exercise 4.12: Floyd's All-Pairs Shortest Path Algorithm**Aim:**

To compute all-pairs shortest paths using Floyd's algorithm and output the complete distance matrix before and after execution.

Python Code:

```
def floyd_warshall(n, edges_list):
    """
    Time Complexity: O(n^3), Space Complexity: O(n^2)
    """
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
    for u, v, w in edges_list:
        dist[u][v] = w

    print("Initial Distance Matrix:")
    for row in dist: print(row)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    print("Shortest Distance Matrix:")
    for row in dist: print(row)
    return dist

# Network of 4 cities (0-indexed: City 1 -> 0, City 2 -> 1, City 3 -> 2, City 4 -> 3)
edges = [(0, 1, 3), (0, 2, 8), (0, 3, -4), (1, 3, 1), (1, 2, 4), (2, 0, 2), (3, 2, -5), (3, 1, 6)]
d = floyd_warshall(4, edges)
print("Shortest path from City 1 to City 3:", d[0][2])
```

Sample Input:

City 1 to City 2: 3, City 1 to City 3: 8, City 1 to City 4: -4, City 2 to 4: 1, City 2 to 3: 4, City 3 to 1: 2, City 4 to 3: -5, City 4 to 2: 6

Sample Output:

Output: City 1 to City 3 = -9

Exercise 4.13: Floyd's Algorithm Router Network Link Failure Simulation

Aim:

To compute shortest routing paths between 6 routers and simulate the impact when the link between Router B and Router D fails.

Python Code:

```
def simulate_link_failure():
    # Routers A-F: 0 to 5
    nodes = ['A', 'B', 'C', 'D', 'E', 'F']
    INF = float('inf')

    def run_floyd(edges):
        dist = [[INF] * 6 for _ in range(6)]
        for i in range(6): dist[i][i] = 0
        for u, v, w in edges:
            dist[u][v] = w
            dist[v][u] = w
        for k in range(6):
            for i in range(6):
                for j in range(6):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
        return dist

    initial_edges = [(0, 1, 1), (0, 2, 5), (1, 2, 2), (1, 3, 1), (2, 4, 3), (3, 4, 1), (3, 5, 6), (4, 5, 2)]
    d_before = run_floyd(initial_edges)
    print("Distance A to F before failure:", d_before[0][5])

    # Link B-D (1-3) fails
    failed_edges = [e for e in initial_edges if not (e[0] == 1 and e[1] == 3)]
    d_after = run_floyd(failed_edges)
    print("Distance A to F after failure: ", d_after[0][5])

    simulate_link_failure()
```

Sample Input:

Router network with 6 nodes A-F. Router B to D fails.

Sample Output:

Output: Router A to Router F = 5 (Path: A -> B -> D -> E -> F: 1+1+1+2=5)

Exercise 4.14: Threshold Distance Reachable Cities (Floyd-Warshall)**Aim:**

To find the city with the smallest number of neighbors reachable within distanceThreshold, breaking ties with the largest city index.

Python Code:

```
def find_the_city(n, edges, distanceThreshold):
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n): dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    min_cnt = float('inf')
    best_city = -1
    for i in range(n):
        cnt = sum(1 for j in range(n) if i != j and dist[i][j] <= distanceThreshold)
        if cnt <= min_cnt:
            min_cnt = cnt
            best_city = i
    return best_city

# Test Case
edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]
print("Resulting City:", find_the_city(5, edges, 2))
```

Sample Input:

n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distanceThreshold = 2

Sample Output:

Output: 0

Exercise 4.15: Optimal Binary Search Tree (OBST) Construction

Aim:

To construct an Optimal Binary Search Tree for keys A, B, C, D with frequencies 0.1, 0.2, 0.4, 0.3, printing cost and root tables.

Python Code:

```
def obst(keys, p):
    n = len(keys)
    e = [[0.0] * (n + 2) for _ in range(n + 2)]
    w = [[0.0] * (n + 2) for _ in range(n + 2)]
    root = [[0] * (n + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        e[i][i - 1] = 0.0
        w[i][i - 1] = 0.0

    for l in range(1, n + 1):
        for i in range(1, n - l + 2):
            j = i + l - 1
            e[i][j] = float('inf')
            w[i][j] = w[i][j - 1] + p[j - 1]
            for r in range(i, j + 1):
                t = e[i][r - 1] + e[r + 1][j] + w[i][j]
                if t < e[i][j]:
                    e[i][j] = round(t, 4)
                    root[i][j] = r

    return e[1][n], e, root

cost, e_table, r_table = obst(['A', 'B', 'C', 'D'], [0.1, 0.2, 0.4, 0.3])
print(f"Optimal Search Cost: {cost}")
```

Sample Input:

N=4, Keys={A,B,C,D}, Frequencies={0.1, 0.2, 0.4, 0.3}

Sample Output:

Optimal Cost: 1.7

Root Table generated with optimal root selections.

Exercise 4.16: OBST with Integer Key Frequencies**Aim:**

To construct an OBST for integer keys {10, 12, 16, 21} with frequencies {4, 2, 6, 3} and display minimum search cost.

Python Code:

```
def obst_integer(keys, freq):
    n = len(keys)
    cost = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for L in range(2, n + 1):
        for i in range(n - L + 1):
            j = i + L - 1
            cost[i][j] = float('inf')
            sum_f = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                c = (cost[i][r - 1] if r > i else 0) + (cost[r + 1][j] if r < j else 0) + sum_f
                cost[i][j] = min(cost[i][j], c)

    return cost[0][n - 1]

print("Keys: [10, 12, 16, 21], Freq: [4, 2, 6, 3] -> Cost:", obst_integer([10,12,16,21], [4,2,6,3]))
```

Sample Input:

Keys = {10, 12, 16, 21}, Frequencies = {4, 2, 6, 3}

Sample Output:

Output: 26

Exercise 4.17: Cat and Mouse Game on Undirected Graph

Aim:

To determine the optimal outcome (1 for mouse win, 2 for cat win, 0 for draw) using minimax game theory DP.

Python Code:

```
from collections import deque

def cat_mouse_game(graph):
    """
    Minimax game on graph.
    Node 0: hole, Mouse starts at 1, Cat starts at 2.
    """
    n = len(graph)
    # color[m][c][turn]
    color = [[[0] * 2 for _ in range(n)] for _ in range(n)]
    degree = [[[0] * 2 for _ in range(n)] for _ in range(n)]

    for m in range(n):
        for c in range(n):
            degree[m][c][0] = len(graph[m])
            degree[m][c][1] = len([nxt for nxt in graph[c] if nxt != 0])

    q = deque()
    # Mouse in hole: Mouse wins (1)
    for c in range(1, n):
        for turn in range(2):
            color[0][c][turn] = 1
            q.append((0, c, turn, 1))
    # Cat catches Mouse: Cat wins (2)
    for m in range(1, n):
        for turn in range(2):
            color[m][m][turn] = 2
            q.append((m, m, turn, 2))

    while q:
        m, c, turn, res = q.popleft()
        prev_turn = 1 - turn
        for prev_m, prev_c in ((prev, c) for prev in graph[m] if prev_turn == 0 else [(m, prev) for prev in graph[c] if prev != 0]):
            if color[prev_m][prev_c][prev_turn] == 0:
                if (prev_turn == 0 and res == 1) or (prev_turn == 1 and res == 2):
                    color[prev_m][prev_c][prev_turn] = res
                    q.append((prev_m, prev_c, prev_turn, res))
            else:
                degree[prev_m][prev_c][prev_turn] -= 1
                if degree[prev_m][prev_c][prev_turn] == 0:
                    color[prev_m][prev_c][prev_turn] = 2 if prev_turn == 0 else 1
                    q.append((prev_m, prev_c, prev_turn, color[prev_m][prev_c][prev_turn]))

    return color[1][2][0]

# Test Cases
print("Graph 1:", cat_mouse_game([[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]))
print("Graph 2:", cat_mouse_game([[1,3],[0],[3],[0,2]]))
```

Sample Input:

Graph 1: [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]
 Graph 2: [[1,3],[0],[3],[0,2]]

Sample Output:

Output: 0 (Draw)

Output: 1 (Mouse wins)

Exercise 4.18: Path with Maximum Probability of Success**Aim:**

To find the path with maximum probability of success between start and end nodes in a weighted graph using modified Dijkstra / DP.

Python Code:

```
import heapq
from collections import defaultdict

def max_probability(n, edges, succProb, start, end):
    graph = defaultdict(list)
    for (u, v), p in zip(edges, succProb):
        graph[u].append((v, p))
        graph[v].append((u, p))

    probs = [0.0] * n
    probs[start] = 1.0
    pq = [(-1.0, start)]

    while pq:
        curr_p, u = heapq.heappop(pq)
        curr_p = -curr_p
        if u == end:
            return f"{curr_p:.5f}"
        if curr_p < probs[u]:
            continue
        for v, p in graph[u]:
            if probs[u] * p > probs[v]:
                probs[v] = probs[u] * p
                heapq.heappush(pq, (-probs[v], v))

    return f"{0.0:.5f}"

# Test Cases
print(max_probability(3, [[0,1],[1,2],[0,2]], [0.5,0.5,0.2], 0, 2))
print(max_probability(3, [[0,1],[1,2],[0,2]], [0.5,0.5,0.3], 0, 2))
```

Sample Input:

n=3, edges=[[0,1],[1,2],[0,2]], succProb=[0.5,0.5,0.2], start=0, end=2
succProb=[0.5,0.5,0.3]

Sample Output:

Output: 0.25000

Output: 0.30000

Exercise 4.19: Unique Paths Grid Counting (DP Table)**Aim:**

To find the number of possible unique paths from grid[0][0] to grid[m-1][n-1] using 2D dynamic programming.

Python Code:

```
def unique_paths_dp(m, n):
    dp = [[1] * n for _ in range(m)]
    for r in range(1, m):
        for c in range(1, n):
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]
    return dp[m - 1][n - 1]

# Test Cases
print("m=3, n=7 ->", unique_paths_dp(3, 7))
print("m=3, n=2 ->", unique_paths_dp(3, 2))
```

Sample Input:

```
m = 3, n = 7
m = 3, n = 2
```

Sample Output:

```
Output: 28
Output: 3
```

Exercise 4.20: Number of Good Pairs**Aim:**

To count the number of good pairs (i, j) such that nums[i] == nums[j] and i < j using frequency counting in O(n) time.

Python Code:

```
from collections import Counter

def num_identical_pairs(nums):
    """
    Formula: for each number with freq k, pairs = k * (k - 1) // 2.
    Time Complexity: O(n), Space Complexity: O(n)
    """
    counts = Counter(nums)
    return sum(v * (v - 1) // 2 for v in counts.values())

# Test Cases
print(num_identical_pairs([1, 2, 3, 1, 1, 3]))
print(num_identical_pairs([1, 1, 1, 1]))
```

Sample Input:

```
nums = [1, 2, 3, 1, 1, 3]
nums = [1, 1, 1, 1]
```

Sample Output:

```
Output: 4
Output: 6
```

Exercise 4.21: City with Smallest Number of Neighbors at Threshold

Aim:

To determine the city with the minimum number of reachable cities within a given threshold distance.

Python Code:

```
def get_best_city(n, edges, thresh):
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n): dist[i][i] = 0
    for u, v, w in edges: dist[u][v] = dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    ans = (-1, float('inf'))
    for i in range(n):
        cnt = sum(1 for j in range(n) if i != j and dist[i][j] <= thresh)
        if cnt <= ans[1]:
            ans = (i, cnt)
    return ans[0]

# Test Cases
print(get_best_city(4, [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], 4))
print(get_best_city(5, [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], 2))
```

Sample Input:

Case 1: n=4, edges=[[0,1,3],[1,2,1],[1,3,4],[2,3,1]], thresh=4

Case 2: n=5, thresh=2

Sample Output:

Output: 3

Output: 0

Exercise 4.22: Network Delay Time (Dijkstra / DP)**Aim:**

To calculate the minimum time for a signal sent from node k to reach all n nodes in a directed weighted network, or return -1 if unreachable.

Python Code:

```
import heapq
from collections import defaultdict

def network_delay_time(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))

    pq = [(0, k)]
    dist = {}

    while pq:
        d, u = heapq.heappop(pq)
        if u in dist:
            continue
        dist[u] = d
        for v, w in graph[u]:
            if v not in dist:
                heapq.heappush(pq, (d + w, v))

    return max(dist.values()) if len(dist) == n else -1

# Test Cases
print(network_delay_time([[2,1,1],[2,3,1],[3,4,1]], 4, 2))
print(network_delay_time([[1,2,1]], 2, 1))
print(network_delay_time([[1,2,1]], 2, 2))
```

Sample Input:

```
times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
times = [[1,2,1]], n = 2, k = 1
times = [[1,2,1]], n = 2, k = 2
```

Sample Output:

```
Output: 2
Output: 1
Output: -1
```